

DEPLOYMENT & SETUP GUIDE

Auto EDA

Automated Exploratory Data Analysis platform — FastAPI backend ·
React + Vite frontend · Multi-cloud import · Docker ready

**Backend**

FastAPI · Python 3.11+

**Frontend**

React 18 · Vite 5

**Docker**

Docker Compose v3.9

**Processing**

Pandas · PySpark

**Cloud**

Azure · AWS · GCP · Databricks

**Database**

PostgreSQL (optional)

Auto EDA Platform · v1.0

Backend :8000 · Frontend :5173 (dev) / :80 (Docker)

May 2026

NAVIGATION

Table of Contents

1

Prerequisites

Software you need to install before starting

2

Get the Source Code

Clone from GitHub or copy the project folder

3

Method A — Run Without Docker (Manual)

Backend Python venv + Frontend Node — full step-by-step

4

Method B — Run With Docker Compose

One command to build and start everything — recommended for deployment

5

Environment Variables Reference

All .env fields for backend and frontend

6

Verify the Installation

Health checks, URLs, and quick smoke test

7

Troubleshooting

Common errors and how to fix them

SECTION 1

Prerequisites

Install the following tools before running Auto EDA. All are free and cross-platform (Windows · macOS · Linux).

Without Docker

TOOL	MINIMUM VERSION	DOWNLOAD	NOTES
Python	3.11 or later	python.org/downloads	Add to PATH during install
pip	comes with Python	—	Run <code>python -m pip install --upgrade pip</code>
Node.js	18 LTS or later	nodejs.org	Includes npm automatically
Git	any recent	git-scm.com	Only needed to clone the repo
Java JRE/JDK	8 or later (optional)	adoptium.net	Only required for PySpark (files ≥ 100 MB). Without Java, Pandas is used as fallback.

With Docker

TOOL	MINIMUM VERSION	DOWNLOAD
Docker Desktop	24.0 or later	docker.com/products/docker-desktop
Docker Compose	v2 (bundled with Docker Desktop)	Included in Docker Desktop



Verify installations

Run these commands after installing. All should print a version number:

```
python --version
node --version
npm --version
docker --version
docker compose version
```

**Windows users**

Use **PowerShell** or **Git Bash** — avoid CMD for path-related commands. When activating the virtual environment use `.\venv\Scripts\activate` (PowerShell) or `venv\Scripts\activate` (CMD).

SECTION 2

Get the Source Code

Clone from GitHub or copy the project folder to your machine.

Option A Clone from GitHub

```
# Clone the repository
git clone https://github.com/ChaudharyPratham/auto_eda.git

# Enter the project directory
cd auto_eda
```

Option B Copy the folder manually

If you have the project as a ZIP or on a USB drive, extract / copy it to your machine. The root folder should look like this:

```
auto_eda/
├── backend/           # FastAPI Python app
│   ├── main.py
│   ├── requirements.txt
│   ├── Dockerfile
│   ├── .env.example
│   ├── routes/
│   ├── services/
│   └── utils/
├── frontend/         # React + Vite app
│   ├── src/
│   ├── package.json
│   ├── Dockerfile
│   └── .env.example
├── docker-compose.yml
├── README.md
└── SETUP.md
```

**Folder name with a space**

The root folder is named `auto eda` (with a space). When typing paths in the terminal, wrap it in quotes:

```
cd "C:\Users\YourName\Desktop\auto eda"
```

SECTION 3 — METHOD A

Run Without Docker

Manual setup using a Python virtual environment and Node.js. Requires two terminal windows running simultaneously.

**Best for**

Local development, debugging, or machines where Docker is not available.

Part 1 — Backend Setup

Open a terminal, navigate to the project root, then run every command below in order.

1**Navigate to the backend folder**

```
# Windows (PowerShell)
cd "C:\path\to\auto eda\backend"

# macOS / Linux
cd "/path/to/auto eda/backend"
```

2**Create a Python virtual environment**

```
python -m venv venv
```

3**Activate the virtual environment**

```
# Windows (PowerShell)
.\venv\Scripts\activate

# macOS / Linux
source venv/bin/activate
```

Your prompt will change to show `(venv)` when active.

4

Install Python dependencies

```
pip install -r requirements.txt
```

This installs FastAPI, Pandas, PySpark, Pillow, cloud SDKs, and more. Takes ~2–5 minutes on first run.

5

Create the .env file

```
# Windows  
copy .env.example .env  
  
# macOS / Linux  
cp .env.example .env
```

The defaults work out of the box. See Section 5 for all configurable fields.

6

Start the backend server

```
python -m uvicorn main:app --reload --port 8000
```

You should see: *Uvicorn running on http://0.0.0.0:8000*. Keep this terminal open.

SECTION 3 — METHOD A (CONTINUED)

Run Without Docker (Frontend)

Open a **second terminal window** — keep the backend terminal running.

Part 2 — Frontend Setup

1

Navigate to the frontend folder

```
# Windows (PowerShell)
cd "C:\path\to\auto eda\frontend"

# macOS / Linux
cd "/path/to/auto eda/frontend"
```

2

Install Node dependencies

```
npm install
```

Downloads ~200 MB of packages into `node_modules/`. Takes ~1–3 minutes.

3

Create the frontend .env file

```
# Windows
copy .env.example .env

# macOS / Linux
cp .env.example .env
```

Leave `VITE_API_URL` empty — the Vite dev proxy automatically forwards `/api/*` to `localhost:8000`.

4

Start the frontend dev server

```
npm run dev
```

You should see: *Local: http://localhost:5173*. Open this URL in your browser.

Access URLs (No Docker)

:5173

Frontend (React App)

<http://localhost:5173>

:8000

Backend API

<http://localhost:8000>

/docs

Swagger UI

<http://localhost:8000/docs>



To stop the servers

Press `Ctrl + C` in each terminal window.

SECTION 4 — METHOD B

Run With Docker Compose

Single command to build and start both backend and frontend. Docker handles all dependencies automatically.

**Best for**

Sharing with teammates, server deployment, or consistent environments. No Python or Node.js installation needed on the host machine.

1**Navigate to the project root folder**

```
# Windows (PowerShell)
cd "C:\path\to\auto eda"

# macOS / Linux
cd "/path/to/auto eda"
```

2**Create the backend .env file**

```
# Windows
copy backend\.env.example backend\.env

# macOS / Linux
cp backend/.env.example backend/.env
```

Edit `backend/.env` to set any optional cloud / database settings (see Section 5).

3**Build and start all containers**

```
docker compose up --build
```

First build takes ~5–10 minutes (downloads base images, installs packages). Subsequent starts are instant. You will see logs from both backend and frontend.

4**Open the app in your browser**

Go to **http://localhost** (port 80). The frontend and backend are both running.

Access URLs (Docker)

:80

Frontend (nginx)
http://localhost

:8000

Backend API
http://localhost:8000

/docs

Swagger UI
http://localhost:8000/docs

Other useful Docker commands

WHAT YOU WANT TO DO	COMMAND
Run in background (detached)	<code>docker compose up --build -d</code>
Stop containers	<code>docker compose down</code>
Stop & remove volumes	<code>docker compose down -v</code>
Rebuild without cache	<code>docker compose build --no-cache</code>
View live logs	<code>docker compose logs -f</code>
View backend logs only	<code>docker compose logs -f backend</code>
Restart a single service	<code>docker compose restart backend</code>
Shell into backend container	<code>docker compose exec backend bash</code>



Make sure Docker Desktop is running

Before running any `docker` command, open Docker Desktop and wait until it shows *Engine running*.

SECTION 5

Environment Variables Reference

All settings live in `backend/.env`. Copy `backend/.env.example` → `backend/.env` and fill in only what you need. Everything marked **OPTIONAL** can be left blank — the server starts fine without it.

Backend — `backend/.env`

```
# — Server —
HOST=0.0.0.0                # Leave as-is
PORT=8000                  # Backend port

# — CORS —
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
# Add your production frontend URL here if deploying

# — Upload size —
MAX_UPLOAD_SIZE_MB=500

# — PostgreSQL (optional) —
DATABASE_URL=postgresql://user:password@localhost:5432/autoeda
# Leave blank to skip database. Format: postgresql://user:pass@host:port/dbname

# — Azure Blob Storage (optional) —
AZURE_STORAGE_CONNECTION_STRING=
AZURE_STORAGE_ACCOUNT_NAME=
AZURE_STORAGE_ACCOUNT_KEY=

# — AWS S3 (optional) —
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_REGION=us-east-1

# — Google Cloud Storage (optional) —
GCP_PROJECT_ID=
GCP_CREDENTIALS_JSON=      # Full path to service-account JSON file

# — Databricks (optional) —
DATABRICKS_HOST=           # e.g. https://adb-1234567890.1.azuredatabricks.net
DATABRICKS_TOKEN=          # Personal access token
DATABRICKS_HTTP_PATH=      # Only needed for SQL Warehouse queries
```

VARIABLE	REQUIRED?	DESCRIPTION
<code>HOST</code>	OPTIONAL	Bind address. Keep <code>0.0.0.0</code> for Docker; use <code>127.0.0.1</code> for local-only.
<code>PORT</code>	OPTIONAL	Backend port. Default 8000.
<code>ALLOWED_ORIGINS</code>	OPTIONAL	Comma-separated list of frontend URLs allowed by CORS.
<code>MAX_UPLOAD_SIZE_MB</code>	OPTIONAL	Max file size for uploads. Default 500 MB.
<code>DATABASE_URL</code>	OPTIONAL	PostgreSQL connection string. Leave blank to run without a database.
<code>AZURE_STORAGE_*</code>	OPTIONAL	Required only if importing data from Azure Blob Storage.
<code>AWS_*</code>	OPTIONAL	Required only if importing data from AWS S3.
<code>GCP_*</code>	OPTIONAL	Required only if importing data from Google Cloud Storage.
<code>DATABRICKS_*</code>	OPTIONAL	Required only if importing data from Databricks DBFS / Volumes.

Frontend — `frontend/.env`

```
# Vite dev proxy forwards /api/* to localhost:8000 automatically.
# Leave VITE_API_URL blank for local development.
VITE_API_URL=
```



Per-dataset API keys

API keys for accessing uploaded datasets are generated automatically on each upload and stored as SHA-256 hashes in `uploads/.keys/`. No manual configuration needed — use the **Generate API Key** button in the Download tab of the dashboard.

SECTION 6

Verify the Installation

Use these checks to confirm everything is running correctly after setup.

1. Health check — Backend API

Open a browser or run in terminal:

```
# In browser, open:
http://localhost:8000/

# Or via curl:
curl http://localhost:8000/

# Expected response:
{"message": "Auto EDA API is running"}
```

2. Swagger interactive docs

Open in browser — you can test all API endpoints here:

```
http://localhost:8000/docs
```

3. Frontend loads

Open the app URL:

```
# No Docker (dev mode):
http://localhost:5173

# With Docker:
http://localhost
```

You should see the Auto EDA landing page with upload options.

4. Quick end-to-end smoke test

☐ Drag and drop a `.csv` file onto the upload area

- ☐ The app navigates to the Dashboard automatically
- ☐ The **Overview** tab shows row/column count and basic stats
- ☐ The **Visualization** tab shows interactive Plotly charts
- ☐ The **Cleaning** tab shows detected issues with a checklist
- ☐ The **Download** tab lets you download cleaned CSV and generate an API key

**All checks passed?**

Auto EDA is fully operational. You can now upload CSV, JSON, Excel, Parquet, Avro, text, image folders, and cloud-hosted data.

SECTION 7

Troubleshooting

Most common errors and how to fix them.

✗ "Could not import module main" when starting uvicorn

You must run uvicorn **from inside the** `backend/` **folder:**

```
cd backend
python -m uvicorn main:app --reload --port 8000
```

Never run it from the project root or with a path like `backend/main:app`.

✗ "ModuleNotFoundError" after activating venv

The venv was not activated before installing, or packages were installed globally instead. Fix:

```
# Make sure venv is active (see (venv) in prompt), then:
pip install -r requirements.txt
```

✗ Frontend shows "Network Error" or blank page

The backend is not running, or the port is wrong. Confirm:

- Backend terminal is still open and shows *Uvicorn running*
- `http://localhost:8000` returns a JSON response in the browser
- No firewall is blocking port 8000

✗ "EADDRINUSE: address already in use :::5173" or :::8000

Another process is using that port. Kill it:

```
# Windows (find and kill port 8000):
netstat -ano | findstr :8000
taskkill /PID <PID from above> /F
```

```
# macOS / Linux:  
lsof -ti :8000 | xargs kill -9
```

✗ Docker: "port is already allocated"

Port 80 or 8000 is used by another container or service. Stop other containers first:

```
docker compose down  
docker ps -a          # list all containers  
docker stop <id>      # stop the conflicting one
```

✗ "PySpark not available" or Java error in logs

Java is not installed or not on PATH. For files under 100 MB Pandas is used as fallback automatically — no action needed. For files $\geq$ 100 MB, install Java:

```
# Check Java:  
java -version  
# If missing, download from adoptium.net and add to PATH.
```

✗ Build error: "Cannot resolve 'buffer/' from Plotly

The frontend imports `plotly.js` (source build) instead of the pre-built dist. Check `VisualizationPanel.jsx`:

```
// ✓ Correct:  
import Plotly from 'plotly.js/dist/plotly'  
  
// ✗ Wrong:  
import Plotly from 'plotly.js'
```

✗ Upload fails with "413 Request Entity Too Large"

Increase `MAX_UPLOAD_SIZE_MB` in `backend/.env`:

```
MAX_UPLOAD_SIZE_MB=1000
```

Then restart the backend server.

✖ Cloud import fails: "azure-storage-blob is not installed"

Cloud SDK packages are listed in `requirements.txt` but may have been skipped. Reinstall with venv active:

```
pip install azure-storage-blob boto3 google-cloud-storage databricks-sdk
```



Need more help?

Check `README.md` and `SETUP.md` in the project root for detailed feature documentation, API reference, and architecture overview.